Table of Contents

Provider..... 1
Resource..... 2
Data Source..... 4
Variables..... 5
Locals..... 8

Important links:
Resouce : resource block reference | Terraform | HashiCorp Developer

Provider

In Terraform, a **Provider** is the essential bridge between your code and the cloud service. Without providers, Terraform doesn't know how to create anything.

The Three Types of Providers

In the Terraform Registry, you will see three tiers:

1. **Official:** Owned and maintained by HashiCorp (e.g., AWS, GCP, Azure).
2. **Verified:** Owned by a third-party vendor but verified by HashiCorp (e.g., MongoDB, Cloudflare, Datadog).
3. **Community:** Maintained by individual developers or small groups.

How to Configure a Provider

There are two parts to using a provider: **Requiring** it and **Configuring** it.

Step 1: Declare the Requirement

This goes in your `terraform {}` block. It tells Terraform exactly where to download the plugin from and which version to use.

```
terraform {
  required_providers {
    aws = {
      source  = "hashicorp/aws"  # Where to get it
      version = "~> 5.0"         # Which version (v5.0 or higher, but
not v6.0)
    }
  }
}
```

Step 2: Configure the Provider

This block sets the settings for that provider, like the region or authentication.

```
provider "aws" {
  region = "us-east-1"
}
```

3. Advanced Concept: Provider Aliases

Sometimes you need to deploy resources to **two different regions** in one script. You do this using an `alias`.

```
# Default provider (US East)
provider "aws" {
  region = "us-east-1"
}

# Alternate provider (US West)
provider "aws" {
  alias  = "west"
  region = "us-west-2"
}

# When creating a resource, tell it which one to use:
resource "aws_instance" "example" {
  provider = aws.west
  ami      = "ami-123456"
  # ...
}
```

Resource

A **resource** is any infrastructure object you want to create and manage with Terraform.

Every resource block follows a strict pattern. Here is a breakdown using an AWS instance as an example:

```
#  (A)        (B)            (C)
resource "aws_instance" "web_server" {
  ami           = "ami-0c55b159cbfafe1f0" # (D) Argument
  instance_type = "t2.micro"              # (D) Argument
}
```

(A) Block Header: Always starts with the word **`resource`**.

(B) Resource Type: This is defined by the provider (e.g., `aws_instance`, `google_compute_instance`). It tells Terraform *what* to create.

(C) Local Name: This is a nickname you give the resource (e.g., `web_server`). You use this name to refer to this resource elsewhere in your code. It does not show up in your cloud console.

(D) Arguments: These are the settings for the resource. Each resource type has different required and optional arguments.

Define `operation` timeouts

Some resource types support a `timeouts` argument that lets you customize how long Terraform waits for certain operations to complete before exiting with a timeout error

```
resource "aws_db_instance" "example" {
  # ...

  timeouts {
    create = "60m"
    delete = "2h"
  }
}
```

`create_before_destroy`

The `create_before_destroy` argument instructs Terraform to create a replacement resource before destroying the resource it replaces.

```
resource {
  lifecycle {
    create_before_destroy = <boolean>
  }
}
```

`prevent_destroy`

The `prevent_destroy` argument instructs Terraform to reject plans to destroy the resource.

```
resource {
  lifecycle {
    prevent_destroy = <boolean>
  }
}

data "provider_type" "local_name" {
# Filters to find the specific item
}
```

Data Source

A **Data Source** is a "Read-Only" operation in Terraform. While a resource tells Terraform to create something, a data block tells Terraform to look up something that already exists.

Imagine you are joining a new company. They already have a network (VPC) set up. You don't want to create a new network; you just want to find its ID so you can put your new server inside it. You use a Data Source to "fetch" that ID.

1. How to use Data Sources

Data sources are defined using the `data` block. The syntax is almost identical to a resource:

```
data "provider_type" "local_name" {
  # Filters to find the specific item
}
```

To use the information fetched, you reference it like this:

```
data.provider_type.local_name.attribute
```

2. Full Working Example: Fetching the Latest AMI

In this example, we don't want to hardcode an Amazon Machine Image (AMI) ID because they change every time Amazon releases a patch. Instead, we'll ask Terraform to find the latest Ubuntu image for us.

```
The Code (main.tf)

provider "aws" {
  region = "us-east-1"
}

# 1. THE DATA SOURCE: Find the latest Ubuntu 22.04 Image
data "aws_ami" "latest_ubuntu" {
  most_recent = true
  owners      = ["033228199246"] # Official Canonical ID

  filter {
    name   = "name"
    values = ["ubuntu/images/hvm-ssd/ubuntu-jammy-22.04-amd64-server-
*"]
  }
}

# 2. THE RESOURCE: Use the ID we just found
resource "aws_instance" "my_server" {
```

```
  ami           = data.aws_ami.latest_ubuntu.id # Fetching ID from
data source
  instance_type = "t2.micro"

  tags = {
    Name = "Web-Server-Found-Via-Data"
  }
}

# 3. THE OUTPUT: Prove it worked
output "ami_id_used" {
  value = data.aws_ami.latest_ubuntu.id
}
```

Variables

In Terraform, **Variables** are like the "**Input Parameters**" of a function. Instead of writing the same values directly into your code (hardcoding), you use variables to make your code flexible and reusable across different environments (like Dev, QA, and Production).

---

1. Why Use Variables?

1. **Reusability:** You can use the same Terraform code to deploy 10 different servers just by changing the input.
2. **Security:** You can keep sensitive data (like API keys) out of your main code files.
3. **Consistency:** Changing a value in one place (the variable file) updates it everywhere that variable is used.

---

2. How to Use Variables (The 3-Step Process)

1. **Define:** Create a `variable` block to tell Terraform the name and type.
2. **Assign:** Provide a value (in a `.tfvars` file, environment variable, or command line).
3. **Reference:** Use the variable in your resources using the `var.<NAME>` syntax.

---

3. Full Working Example: Docker Provider

In this example, we will create a Docker container. We will use variables to decide which **image** to use and what **external port** to open.

Step 1: Define the Variables (`variables.tf`)

```
variable "container_name" {
  description = "Name of the docker container"
  type        = string
  default     = "my-web-app"
}

variable "image_name" {
  description = "Docker image to use"
  type        = string
  default     = "nginx:latest"
}

variable "external_port" {
  description = "Port to expose on the host"
  type        = number
  default     = 8080
}
```

Step 2: Use the Variables (main.tf)

```
terraform {
  required_providers {
    docker = {
      source  = "kreuzwerker/docker"
      version = "~> 3.0.1"
    }
  }
}

provider "docker" {}

# Download the image
resource "docker_image" "app_image" {
  name = var.image_name # Referencing the variable
}

# Start the container
resource "docker_container" "app_container" {
  image = docker_image.app_image.image_id
  name  = var.container_name # Referencing the variable

  ports {
    internal = 80
    external = var.external_port # Referencing the variable
  }
}
```

Step 3: Assign Values (`terraform.tfvars`)

If you want to override the defaults, create this file. Terraform automatically loads it.

```
external_port  = 9000
container_name = "production-server"
```

- **Precedence (The Hierarchy):** If you set a variable in multiple places, Terraform follows this order:
    1. Command line (`-var="port=80"`) **(Highest Priority)**
    2. `.tfvars` files
    3. Environment variables (`TF_VAR_port`)
    4. Default values in the code **(Lowest Priority)**

- **Sensitive Variables:** For passwords, set `sensitive = true` in the variable block. This prevents Terraform from printing the password in your terminal during an `apply`.

Variable definition files

```
instance_type     = "t3.large"
environment      = "prod"
subnet_ids       = ["subnet-12345", "subnet-67890", "subnet-abcdef"]
enable_monitoring = true
```

Environment variables

```
export TF_VAR_instance_type=t3.medium
export TF_VAR_environment=staging
terraform apply
```

Command-line variables

```
terraform apply -var="instance_type=t3.medium"
-var="environment=prod"
terraform apply -var='subnet_ids=["subnet-12345","subnet-67890"]'
```

```
terraform apply -var-file="production.auto.tfvars"
```

Locals

In Terraform, a **Local Value** (or "Local") is like a temporary variable in a programming language. While **Variables** are inputs that a user provides to your script, **Locals** are values that you calculate *inside* your script to keep it clean.

---

1. Why use Locals?

1. **Avoid Repetition:** If you have a complex naming convention (e.g., project-env-service-region), you don't want to type that formula 50 times.
2. **Readability:** You can give a simple name to a complex calculation or function.
3. **Clean Code:** It keeps your resource blocks focused on the "what" rather than the "how."

---

2. Full Flow Example

In this example, we will use variables (inputs) to calculate a "Naming Standard" and "Common Tags" using a Local block.

Step 1: Define Inputs (variables.tf)

```
variable "project" {
  default = "rocket-ship"
}

variable "environment" {
  default = "prod"
}
```

Step 2: Define Logic (locals.tf)

You define them in a locals (plural) block, but you reference them with local (singular).

```
locals {
  # 1. Combining variables into a single name
  name_prefix = "${var.project}-${var.environment}"

  # 2. Creating a standard map of tags
  common_tags = {
    Project   = var.project
    Env       = var.environment
    ManagedBy = "Terraform"
```

```
  }

  # 3. Logic based on environment
  instance_type = var.environment == "prod" ? "t3.medium" :
"t2.micro"
}
```

Step 3: Use the Locals (main.tf)

```
resource "aws_instance" "web" {
  ami           = "ami-12345678"
  instance_type = local.instance_type # Using the logic from locals

  tags = merge(local.common_tags, {
    Name = "${local.name_prefix}-web-server" # Using the name prefix
  })
}
```

Outputs

In Terraform, **Outputs** are like the "return values" of a function. While variables are your inputs, outputs are how your script tells you (and other tools) the important information about what it just built.

2. Full Flow Example: The AWS "Web Server"

In this example, we will build a server and output its connection details.

Step 1: Define the Resource (main.tf)

```
provider "aws" {
  region = "us-east-1"
}

resource "aws_instance" "web_server" {
  ami           = "ami-0c55b159cbfafe1f0"
  instance_type = "t2.micro"

  tags = {
    Name = "Web-Server-Demo"
  }
}
```

Step 2: Define the Outputs (outputs.tf)
It is a best practice to keep your outputs in a separate file.

```
# 1. Basic string output
output "instance_public_ip" {
  description = "The public IP of our server"
  value       = aws_instance.web_server.public_ip
}

# 2. Complex object output (Grouping data)
output "server_details" {
  value = {
    id   = aws_instance.web_server.id
    arn  = aws_instance.web_server.arn
  }
}

# 3. Sensitive output (Redacted from console)
```

```
output "db_password" {
  value     = "super-secret-password-123"
  sensitive = true
}
```

---

3. How to See the Outputs

After you run `terraform apply`, Terraform will print these values at the very bottom of your screen.

If you want to see them again later without re-running everything, use these commands:

- **`terraform output`**: Shows all outputs.
- **`terraform output instance_public_ip`**: Shows just that specific value.
- ***`terraform output -json`***: Returns all data in JSON format—perfect for Python or Bash scripts!

---

4. Pro Tips for Administrators

- **Sensitive Flag:** If an output contains a password or API key, always set `sensitive = true`. This prevents the value from being printed in logs, though it is **still stored** in the state file.
- **Descriptions:** Always include the `description` field. It acts as documentation for your teammates so they know what the output represents.
- **Data Transformation:** You can use functions in outputs. For example:

  ```
  value = upper(aws_instance.web_server.instance_state)
  ```